

Software Requirements Specification (SRS)

Source: docs/SRS.md • Status: Verified & Synchronized

Software Requirements Specification (SRS)

Product: Cubyntra

Organization: Necookie Labs

Standard: Inspired by IEEE Std 830 / ISO/IEC/IEEE 29148

Document Version: 1.0

Status: V1 Release

Last Updated: 2026-09-26

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) establishes the complete functional, algorithmic, architectural, and quality requirements for Cubyntra V1, an autonomous, client-side Computer Vision Rubik's Cube scanner, state validator, solver, and interactive 3D digital twin developed by Necookie Labs.

1.2 Scope

Cubyntra provides client-side video capture, 3×3 adaptive grid sampling, lighting-tolerant color classification, temporal stability buffering, strict physical cube validation (parity, flips, twists), deterministic two-phase solving (Herbert Kociemba algorithm), and high-fidelity 3D layer visualization with zero floating-point drift.

Hardware robotics (ESP32 controllers, stepper motors, servos, and physical grippers) are explicitly marked as FUTURE / OUT OF SCOPE FOR V1.

1.3 Definitions, Acronyms, and Abbreviations

- HTM: Half Turn Metric. Standard notation where any turn of any face by 90° or 180° counts as one move (R, R', R2 = 1 move).
- QTM: Quarter Turn Metric. Metric where a 180° turn counts as two 90° moves.
- ROI: Region of Interest. The bounding box within the video frame where sampling takes place.
- CIELAB: $L^*a^*b^*$ color space defined by the International Commission on Illumination, approximating human perceptual color difference (Delta-E).
- HSV: Hue, Saturation, Value cylindrical-coordinate color representation.
- Two-Phase Algorithm: Herbert Kociemba's algorithm that transitions an arbitrary cube state into subgroup $H = \langle U, D, L2, R2, F2, B2 \rangle$ (Phase 1) and then solves it within H (Phase 2).
- Cubie: An individual physical block of a Rubik's Cube (8 corners, 12 edges, 6 centers, 1 core).

2. Overall Description

2.1 Product Perspective

Cubyntra is a standalone client-side Progressive Web Application (PWA). It does not require any cloud processing, external APIs, authentication gateways, or remote storage.

Cubyntra • Necookie Labs • See it. Solve it.

```

graph TD
    User([User])
    subgraph Browser_Sandbox [Client Browser Sandbox]
        Camera[Camera Feed / MediaDevices] --> Canvas[2D Canvas & Sampling]
        Canvas --> CV[Color Classifier & Stability]
        CV --> Recon[State Reconstruction]
        Recon --> Validator[Parity & Invariant Validator]
        Validator --> Solver[Kociemba Two-Phase Engine]
        Solver --> Moves[CubeMove[] Sequence]
        Moves --> Engine[Three.js Digital Twin]
        Engine --> Visual[60 FPS WebGL Canvas]
        Visual --> User
    end
end

```

2.2 Operating Environment

- Modern desktop and mobile web browsers supporting:
- ECMAScript 2022+ / WebAssembly
- WebRTC (`navigator.mediaDevices.getUserMedia`)
- HTML5 Canvas 2D (`CanvasRenderingContext2D`)
- WebGL 2.0 / Three.js
- Supported Platforms:
- Google Chrome (macOS, Windows, Linux, Android)
- Apple Safari (iOS 16+, iPadOS, macOS)
- Mozilla Firefox (Desktop)
- Microsoft Edge (Windows, macOS)

2.3 Design & Implementation Constraints

- Strict Client-Side Privacy: Raw frames and pixel data must never be transmitted across network sockets.
- Single-Threaded Safety: Heavy computations must be bounded to prevent UI thread freezing.
- Zero Floating-Point Accumulation Drift: Layer rotation matrices must snap cubie quaternions back to orthogonal integer coordinates after every move.

3. Specific Requirements

3.1 Camera Interface Requirements

- FR-CAM-001: The system shall request user permission to access camera video streams before initializing scanning.
- FR-CAM-002: The system shall request `facingMode`: { ideal: 'environment' } on mobile devices to favor rear cameras.
- FR-CAM-003: The system shall release all hardware camera tracks immediately when scanning concludes or the component unmounts.
- FR-CAM-004: In the event of camera permission denial or missing hardware, the system shall display an informative fallback with simulated scramble demonstration mode.

3.2 Computer Vision Requirements

- FR-CV-001: The system shall calculate a centered square Region of Interest (ROI) occupying 65% of the minimum video dimension.
- FR-CV-002: The system shall partition the ROI into a 3 x 3 grid of 9 sampling cells.
- FR-CV-003: The system shall extract pixels from an inner central sub-region (40% area) of each cell to exclude black plastic borders and specular corner reflections.
- FR-CV-004: The system shall compute a trimmed mean of sampled pixels, discarding the upper and lower 15% outliers in R, G, and B channels.
- FR-CV-005: The system shall convert aggregated RGB to HSV and CIELAB color spaces.
- FR-CV-006: The system shall classify each cell into one of the 6 canonical colors: white, yellow, green, blue, red, orange.
- FR-CV-007: The system shall compute confidence metrics based on candidate score margins.
- FR-CV-008: The system shall maintain a rolling temporal stability buffer requiring 8 consecutive frames of identical classification before signaling a stable lock.

3.3 Guided Scanning Requirements

- FR-SCAN-001: The system shall guide the user through capturing 6 unique faces in canonical order: Up (White), Front (Green), Right (Red), Back (Blue), Left (Orange), Down (Yellow).
- FR-SCAN-002: The system shall display physical cube rotation hints between capture steps.
- FR-SCAN-003: The system shall display mini 3x3 thumbnails of all captured faces and permit one-click rescan of any individual face.

3.4 Cube Domain & Validation Requirements

- FR-CUBE-001: The system shall validate that exactly 54 stickers are present with exactly 9 stickers per canonical color.
- FR-CUBE-002: The system shall validate that 6 unique center stickers exist in canonical Western orientation (U=white, R=red, F=green, D=yellow, L=orange, B=blue).
- FR-CUBE-003: The system shall reject states with physically impossible corner pieces (e.g. opposing colors such as White-Yellow on the same corner).
- FR-CUBE-004: The system shall reject states with corner twist parity violations: $\sum_{i=1}^8 \text{twist}_i \not\equiv 0 \pmod{3}$.
- FR-CUBE-005: The system shall reject states with edge flip parity violations: $\sum_{i=1}^{12} \text{flip}_i \not\equiv 0 \pmod{2}$.
- FR-CUBE-006: The system shall reject states with permutation parity mismatches between corners and edges.

3.5 Solver Engine Requirements

- FR-SOLVER-001: The system shall reject invalid cube states before initiating two-phase search.
- FR-SOLVER-002: The system shall return a sequence of structured CubeMove objects adhering to Half Turn Metric (HTM).
- FR-SOLVER-003: The system shall simulate the generated move sequence on an internal logical model and verify that the resulting state is 100% solved before exposing the solution to the user.

3.6 3D Digital Twin Requirements

- FR-3D-001: The system shall construct 27 individual cubies positioned at integer coordinates x, y, z in $\{-1, 0, 1\}$.
- FR-3D-002: The system shall map physical scanned facelet colors to exterior cubie sticker meshes.
- FR-3D-003: The system shall smoothly animate layer rotations using cubic easing.
- FR-3D-004: Upon animation completion, the system shall snap cubie positions to integer coordinates and quaternions to orthogonal angles, maintaining 0.000% accumulation drift.
- FR-3D-005: The system shall render 3D curved directional arrows pointing in the direction of the turn.
- FR-3D-006: The system shall support orbit rotation, pinch/wheel zoom, and view reset.

3.7 Playback & User Interface Requirements

- FR-UI-001: The system shall provide Step Next, Step Previous, Play/Pause, Reset to Scramble, and Speed controls (0.5x, 1x, 2x, 4x).
- FR-UI-002: The system shall display an interactive move ticker highlighting the current active move.
- FR-UI-003: The system shall provide keyboard navigation shortcuts (Space for play/pause, Left/Right arrows for step, R for reset).
- FR-UI-004: The system shall display an optional developer CV Debugger telemetry panel inspecting real-time RGB/HSV and stability counters.

4. Non-Functional Requirements

4.1 Performance Requirements

- NFR-PERF-001: Interactive Three.js rendering shall maintain 60 FPS on modern hardware.
- NFR-PERF-002: Solver calculation time shall complete in under 100ms for standard random scrambles.
- NFR-PERF-003: Video sampling loop shall not trigger excessive React re-renders.

4.2 Security & Privacy Requirements

- NFR-PRIV-001: No webcam video frames or image data shall leave the browser sandbox.
- NFR-SEC-001: Application shall not require API keys or credentials on the client.

4.3 Accessibility Requirements

- NFR-A11Y-001: All controls shall include ARIA labels and visible focus rings.
- NFR-A11Y-002: Procedural background animations shall disable motion when prefers-reduced-motion: reduce is active.

5. Requirements Traceability Matrix

Requirement ID	Description	Source Module	Test Suite

FR-CAM-001..004	Camera Lifecycle & Fallbacks	src/vision/camera.ts	Manual / Browser
FR-CV-001..004	ROI & Trimmed Aggregation	src/vision/sampling.ts	tests/vision.test.ts
FR-CV-005..007	HSV/CIELAB Classification	src/vision/color.ts	tests/vision.test.ts
FR-CV-008	Temporal Stability Buffer	src/vision/stability.ts	tests/vision.test.ts
FR-SCAN-001..003	6-Face Sequence & Rescan	src/components/camera/ CameraScanner.tsx	tests/integration.test.ts
FR-CUBE-001..006	Mathematical Validation & Parities	src/cube/validator.ts	tests/cube.test.ts
FR-SOLVER-001..003	Kociemba Two-Phase Solver	src/solver/solver.ts	tests/solver.test.ts
FR-3D-001..006	3D Twin & Zero-Drift Rotation	src/three/engine.ts	tests/integration.test.ts
FR-UI-001..004	Playback Controls & CV Debugger	src/components/solver/ SolveControls.tsx	tests/integration.test.ts